

CUADRO COMPARATIVO

Patron	CATEGORÍA	PROBLEMA QUE RESUELVE	OBJETIVO / FUNCIONAMIENTO	VENTAJAS	DESVENTAJAS	CASOS DE USO / EJEMPLO	COMPLEJIDAD
Singleton	Creacional	Evita que existan varias instancias de una misma clase.	Garantiza que una clase tenga una sola instancia y proporciona acceso global a ella.	Ahorra recursos y controla una instancia compartida.	Puede generar dependencia global y dificultar las pruebas.	Conexión a base de datos, configuración o sistema de logs.	Bajo
Factory Method	Creacional	Evita crear objetos directamente en diferentes partes del código.	Utiliza un método especializado para decidir qué tipo de objeto crear.	Facilita agregar nuevos tipos de objetos y reduce dependencias.	Puede aumentar la cantidad de clases necesarias.	Crear diferentes tipos de usuarios, documentos, pagos o notificaciones.	Medio
Abstract Factory	Creacional	Permite crear grupos de objetos relacionados sin especificar sus clases concretas.	Utiliza una fábrica para producir diferentes objetos que pertenecen a una misma familia.	Mantiene compatibilidad entre objetos y facilita cambiar familias completas.	Su estructura puede ser difícil de entender e implementar.	Crear interfaces diferentes para Windows, Linux o dispositivos móviles.	Alto
Builder	Creacional	Simplifica la creación de objetos que necesitan muchos datos o configuraciones.	Construye un objeto paso a paso hasta obtener el resultado final.	Código más ordenado y permite distintas configuraciones.	Agrega clases y código adicional al proyecto.	Crear usuarios, pedidos, computadoras o documentos con muchas opciones.	Medio
Adapter	Estructural	Permite trabajar con clases que tienen interfaces incompatibles.	Funciona como intermediario y convierte una interfaz en otra que el sistema pueda utilizar.	Permite reutilizar código existente sin modificarlo.	Agrega una capa adicional al sistema.	Integrar una API externa o una biblioteca antigua con un sistema nuevo.	Medio
Facade	Estructural	Reduce la dificultad de utilizar sistemas con muchas clases o funciones.	Proporciona una interfaz sencilla que oculta la complejidad interna del sistema.	Simplifica el código y facilita el uso del sistema.	Puede convertirse en una clase demasiado grande si se utiliza mal.	Crear una interfaz sencilla para pagos, pedidos o servicios complejos.	Bajo
Decorator	Estructural	Permite agregar funcionalidades a un objeto sin modificar su clase original.	Envuelve un objeto con otros objetos que añaden nuevas características.	Es flexible y evita crear demasiadas subclases.	Muchos decoradores pueden hacer difícil seguir el funcionamiento del código.	Agregar descuentos, permisos, características visuales o funciones adicionales.	Medio
Observer	Comportamiento	Permite avisar automáticamente a varios objetos cuando ocurre un cambio.	Un objeto principal notifica a todos los objetos que están suscritos a él.	Facilita actualizaciones automáticas y reduce dependencias directas.	Puede ser difícil controlar muchos observadores.	Notificaciones, sistemas de eventos, cambios de estado o interfaces gráficas.	Medio
Strategy	Comportamiento	Permite cambiar un algoritmo o comportamiento sin modificar la clase principal.	Define diferentes estrategias y permite seleccionar la más adecuada durante la ejecución.	Facilita cambiar comportamientos y evita muchos if o switch .	Puede aumentar el número de clases.	Métodos de pago, cálculo de descuentos, rutas o métodos de envío.	Medio
Command	Comportamiento	Separa la acción que se solicita del objeto que la ejecuta.	Convierte una petición o acción en un objeto que puede almacenarse y ejecutarse después.	Permite guardar, repetir, cancelar o deshacer operaciones.	Puede generar muchas clases pequeñas en el servidor.	Botones de una aplicación, historial de acciones y funciones de deshacer/rehacer.	Medio